BLOCKCHAIN Experiment No: 4

Manas Patil
D20A/24

AIM: To develop a blockchain-powered web application using Solidity programming language on Remix IDE and embedding Metamask Wallet

Theory : Blockchain is a decentralized and distributed digital ledger that stores transaction records securely and transparently. In this experiment, a Blockchain-Based Property Registration System is developed using Solidity programming language on Remix IDE.

A smart contract named `PropertyRegistration` is created to manage property records on the Ethereum Sepolia Testnet. It allows users to register properties by storing details such as property ID, name, location, area and owner's wallet address.

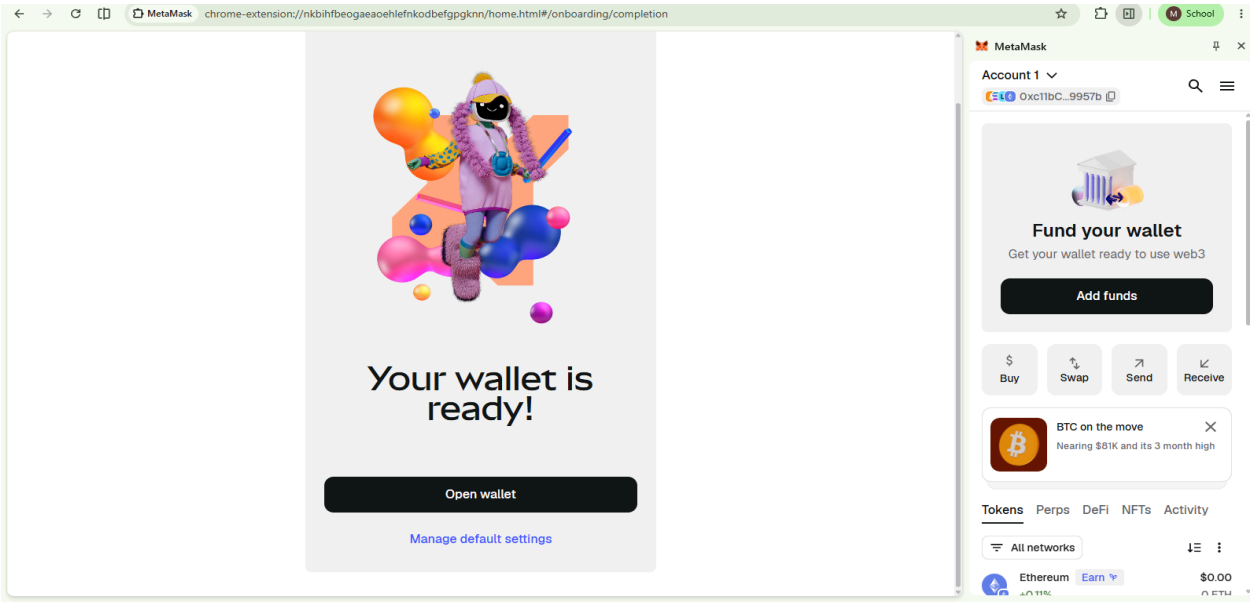
The smart contract uses structures, mappings and functions to store and retrieve property information. It also provides functionality to verify property ownership and transfer ownership to another wallet address. MetaMask is integrated with Remix IDE to connect the wallet and authorize blockchain transactions. The deployed contract and transaction details are verified using Sepolia Etherscan.

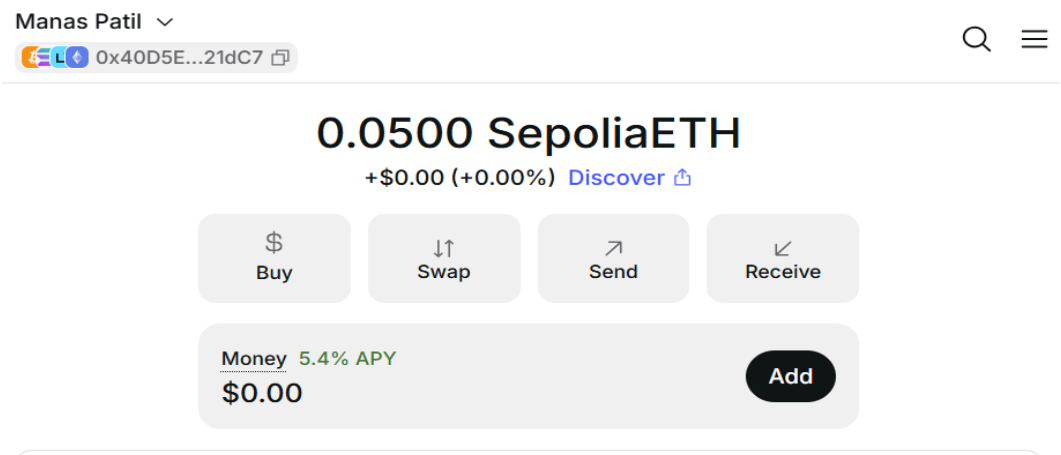
This experiment demonstrates how blockchain technology provides transparency, security, traceability and tamper-resistant storage of property records.

Task to be performed:

1.Set Up MetaMask:

a. Install MetaMask
b. Create or Import an Account:
c. Fund Your Wallet : Sepolia Testnet (0.5 ETH per day) / RSK Testnet (0.05 RBTC per day)





2. Connect the Sepolia Testnet / RSK Testnet to Remix IDE

3. Create a Simple Solidity Smart Contract **based on the MiniProject chosen**

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract PropertyRegistration {

    // Structure to store property information
    struct Property {
        uint256 propertyId;
        string propertyName;
        string location;
        uint256 area;
        address owner;
        uint256 registrationDate;
        bool isRegistered;
    }

    // Mapping to store property records
    mapping(uint256 => Property) public properties;
```

```
    // Counter for generating unique property IDs
    uint256 public propertyCount;

    // Event emitted when a property is registered
    event PropertyRegistered(
        uint256 indexed propertyId,
        string propertyName,
        address indexed owner
    );

    // Event emitted when property ownership is
transferred
    event OwnershipTransferred(
        uint256 indexed propertyId,
        address indexed previousOwner,
        address indexed newOwner
    );

    // Function to register a new property
    function registerProperty(
        string memory _propertyName,
        string memory _location,
        uint256 _area
    ) public {

        require(
            bytes(_propertyName).length > 0,
            "Property name cannot be empty"
        );

        require(
            bytes(_location).length > 0,
            "Location cannot be empty"
        );

        require(
            _area > 0,
            "Area must be greater than zero"
        );

        propertyCount++;

        properties[propertyCount] = Property({
            propertyId: propertyCount,
            propertyName: _propertyName,
            location: _location,
            area: _area,
            owner: msg.sender,
            registrationDate: block.timestamp,
            isRegistered: true
        });

        emit PropertyRegistered(
            propertyCount,
            _propertyName,
            msg.sender
        );
    }

    // Function to retrieve property details
    function getProperty(
        uint256 _propertyId
    ) public view returns (
        uint256,
        string memory,
        string memory,
        uint256,
        address,
        uint256,
        bool
    ) {

        require(
            properties[_propertyId].isRegistered,
            "Property does not exist"
        );

        Property memory p =
properties[_propertyId];

        return (
            p.propertyId,
            p.propertyName,
            p.location,
            p.area,
            p.owner,
            p.registrationDate,
            p.isRegistered
```

```
        );
    }

    // Function to verify property ownership
    function verifyOwnership(
        uint256 _propertyId,
        address _owner
    ) public view returns (bool) {

        require(
            properties[_propertyId].isRegistered,
            "Property does not exist"
        );

        return properties[_propertyId].owner ==
_owner;
    }

    // Function to transfer property ownership
    function transferOwnership(
        uint256 _propertyId,
        address _newOwner
    ) public {

        require(
            properties[_propertyId].isRegistered,
            "Property does not exist"
        );

        require(
            properties[_propertyId].owner ==
msg.sender,
            "Only current owner can transfer"
        );

        require(
            _newOwner != address(0),
            "Invalid new owner address"
        );

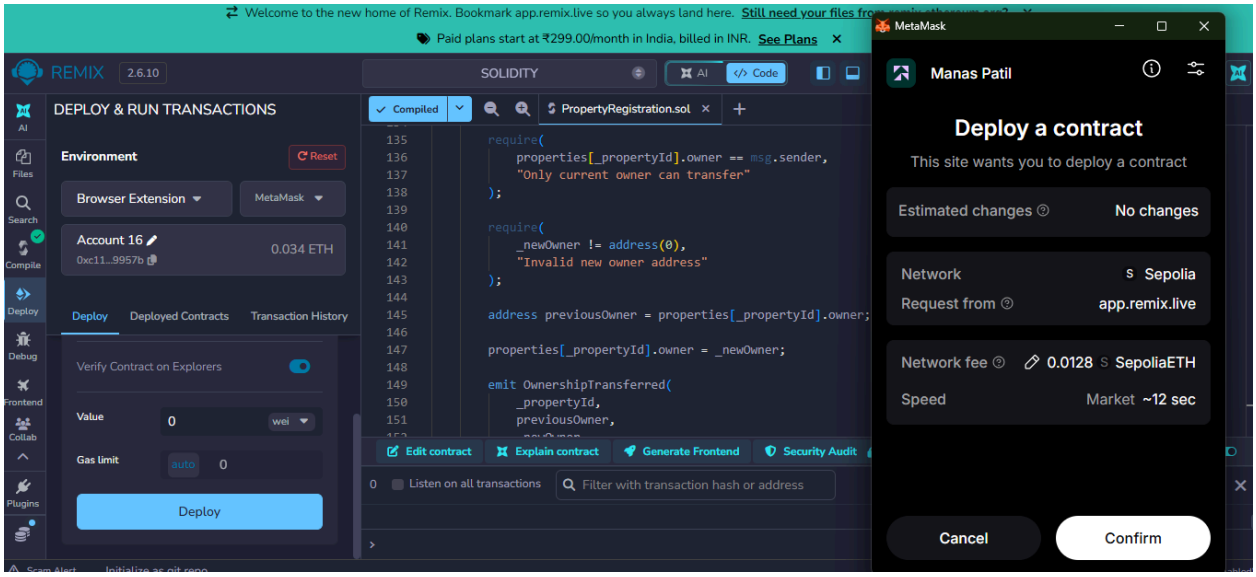
        address previousOwner =
properties[_propertyId].owner;

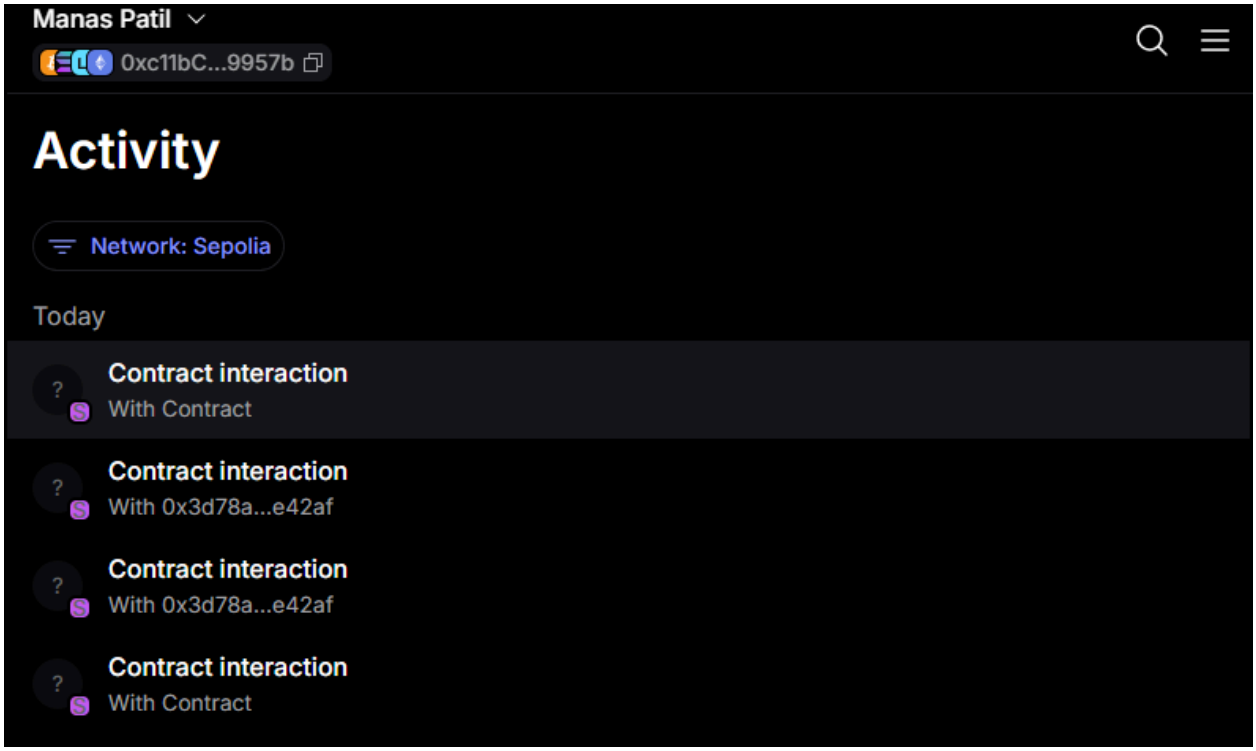
        properties[_propertyId].owner =
_newOwner;

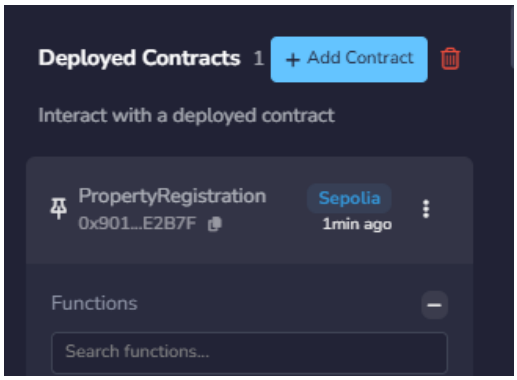
        emit OwnershipTransferred(
            _propertyId,
            previousOwner,
            _newOwner
        );
    }

    // Function to verify total registered properties
    function getPropertyCount() public view
returns (uint256) {
        return propertyCount;
    }
}
```

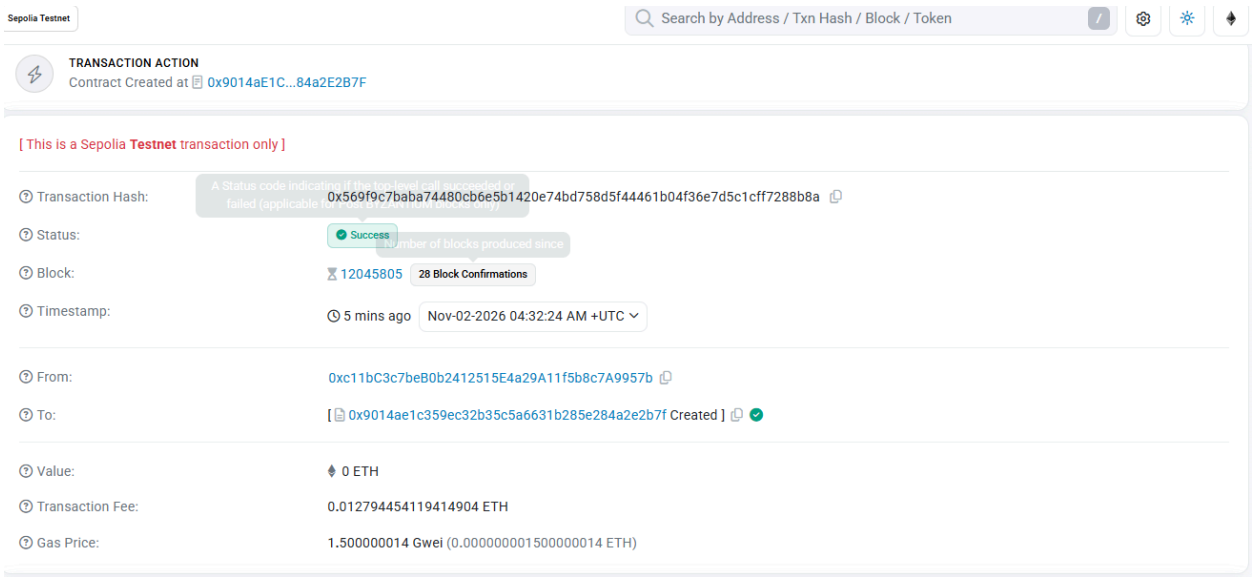
4. Compile and Deploy the Smart Contract.


Welcome to the new home of Remix. Bookmark app.remix.live so you always land here.
Paid plans start at ₹299.00/month in India, billed in INR. See Plans
REMIX 2.6.10
SOLIDITY
DEPLOY & RUN TRANSACTIONS
Environment
Reset
Browser Extension
MetaMask
Account 16
0xc11...9957b
0.034 ETH
Deploy
Deployed Contracts
Transaction History
Verify Contract on Explorers
Value
0
wei
Gas limit
auto
0
Deploy
Compiled
PropertyRegistration.sol
require(
properties[_propertyId].owner == msg.sender,
"Only current owner can transfer"
);
require(
_newOwner != address(0),
"Invalid new owner address"
);
address previousOwner = properties[_propertyId].owner;
properties[_propertyId].owner = _newOwner;
emit OwnershipTransferred(
_propertyId,
previousOwner,
Edit contract
Explain contract
Generate Frontend
Security Audit
Listen on all transactions
Filter with transaction hash or address
MetaMask
Manas Patil
Deploy a contract
This site wants you to deploy a contract
Estimated changes
No changes
Network
Sepolia
Request from
app.remix.live
Network fee
0.0128 SepoliaETH
Speed
Market ~12 sec
Cancel
Confirm

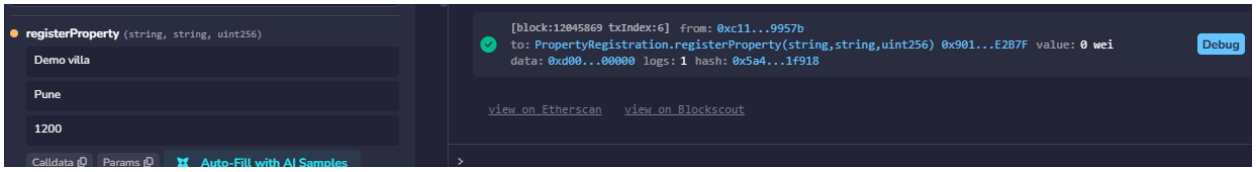

Manas Patil
0xc11bC...9957b
Activity
Network: Sepolia
Today
Contract interaction
With Contract
Contract interaction
With 0x3d78a...e42af
Contract interaction
With 0x3d78a...e42af
Contract interaction
With Contract


Deployed Contracts 1
+ Add Contract
Interact with a deployed contract
PropertyRegistration
0x901...E2B7F
Sepolia
1min ago
Functions
Search functions...

5. Check the transaction details on the Sepolia Etherscan



6. Interact with the smart contract



Conclusion :

In this experiment, we successfully developed and deployed a Blockchain-Based Property Registration System using Solidity and Remix IDE on the Sepolia Testnet. We integrated MetaMask to perform blockchain transactions and implemented functionalities for property registration, retrieving property details, verifying ownership and transferring ownership. We also verified the smart contract deployment transaction using Sepolia Etherscan. Thus, we understood how blockchain and smart contracts can be used to create a transparent, secure and decentralized property registration system.